

符玄桌面宠物 — 技术文档

符玄桌宠项目

N40 • 2026-08-08

摘要

本文档为符玄桌面宠物项目的完整技术报告（N40，2026-08-08 按代码真相审计修订）。项目基于 Tuanjie 2022.3.62t7（团结引擎）与 Live2D Cubism SDK 5-r.4，构建了一个运行于 Windows 桌面的 AI 驱动虚拟角色。系统深度集成 DeepSeek Chat API（含 Function Calling 55 个工具）、GLM-4V 多模态视觉模型、Ollama 本地 LLM 实时决策引擎、本地知识库（RAG）以及多层级感知与记忆系统。核心亮点是**具身智能闭环**——从感知、决策、执行到视觉验证与记忆学习的完整循环。

N39（2026-08-02）已修复全部 9 个审计 Bug（IsSleepTime 真实睡眠判断、ExecuteCombo 锁时序、农历月相、交互时间接线、DualModelValidator 清理、AutoMotionCollector 死代码删除、示例角度修正、反思机制接线、知识库缓存注入）。N40（2026-08-07）完成 T1-T8 Token 优化：工具集 52→55 全量核实、工具子集按意图注入（55→27）、历史 15000 字符预算 + Ollama 摘要、SystemPrompt -41%、Speculative Multi-Action、usage 日志。工具回环为 10 轮（MAX_TOOL_ROUNDS=10），MotionTranslator 规则为 10 条 + 10 种 SPECIAL PATTERNS（9 姿势，捂脸重复），天气源默认 wttr.in（QWeather 可选）。详见第 13 章。

目录

1	项目概述	4
1.1	项目背景	4
1.2	技术栈	4
1.3	核心指标	4
2	系统架构	5
2.1	分层架构	5
2.2	执行顺序	6
2.3	依赖注入模式	6
3	渲染管道	6
3.1	Live2D Cubism 渲染	6
3.1.1	模型加载双保险	6
3.1.2	参数映射系统	6
3.1.3	Perlin 噪声驱动空闲微动	7
3.1.4	天气表情联动	7

目录	2
3.2 DWM 透明窗口	7
4 AI 对话系统	8
4.1 DeepSeek Chat API 集成	8
4.1.1 系统提示词注入	8
4.1.2 Function Calling 架构	8
4.1.3 多轮工具调用循环	8
4.1.4 异步工具执行	9
4.2 工具索引	9
4.3 句子队列与优先级气泡	10
4.4 自动闲聊系统	11
5 具身动作系统	11
5.1 架构演进	11
5.2 空闲动作系统	11
5.2.1 JSON 配置驱动	11
5.2.2 动作列表	12
5.2.3 插值曲线	12
5.3 LLM 动作翻译 (MotionTranslator)	13
5.3.1 核心流程	13
5.3.2 SPECIAL PATTERNS	13
5.4 动作锁定机制	14
5.5 MotionAgent — 自主动作决策引擎	14
5.6 MotionPlanner — 动作规划器	14
5.7 安全与 GPU 监控	15
5.7.1 SafetyValidator — 参数安全校验	15
5.7.2 GpuLoadMonitor — 游戏检测	15
6 闭环具身智能验证	15
6.1 架构演进	15
6.2 验证流程	15
6.3 VisionMotionVerifier — GLM-4V 多帧拼图评分	16
6.4 MotionMemoryManager — 运动记忆引擎（闭环学习核心）	16
6.5 具身验证工具集	16
7 交互系统	17
7.1 拖拽系统 (DragHandler)	17
7.2 悬浮球辐射菜单 (BallPanel)	17
7.3 右侧 Widgets 面板 (RightPanel)	18
7.4 分区点击反馈	18

7.5 拖拽挣扎动画	18
8 感知与记忆系统	18
8.1 法眼活动追踪 (ActivityTracker)	18
8.2 长期记忆系统 (PetMemory)	19
8.3 人格演化系统 (PersonalityManager)	19
8.4 时间与天气系统 (TimeWeatherController)	19
8.4.1 AI 天气语录	19
8.5 本地知识库 (藏书阁) (KnowledgeBaseManager)	19
8.6 核心物理模型	20
8.7 地面任务状态机	20
8.8 多显示器支持	20
8.9 睡眠唤醒检测	20
9 窗口与系统集成	20
9.1 崩溃日志系统	20
9.2 进程互斥	20
10 性能优化	21
10.1 三级性能档位	21
10.2 CPU/GPU/内存监控	21
11 便签与提醒系统	21
12 构建与部署	21
13 已知问题与 Bug (N38 审计 · N39 已全部修复)	21
14 版本路线图	24
15 附录	25
15.1 环境变量清单	25
15.2 相关文件路径	25
15.3 关键设计决策	26

1 项目概述

1.1 项目背景

原桌面宠物程序基于 Windows GDI+ 分层窗口技术，使用静态 PNG 图片硬切换模拟动画。本项目将其迁移至 Unity 引擎，利用 Live2D Cubism SDK 实现参数化变形动画，获得更流畅的视觉效果和更丰富的交互能力，并引入 AI 对话、环境感知、具身动作生成等高级特性。最新版本 V2.4 在 V2.3+ 基础上新增了藏书阁本地知识库 RAG 系统 (KnowledgeBaseManager)，实现了语义检索、文件索引与对话关联搜索等能力。

1.2 技术栈

- 引擎: Tuanjie 2022.3.62t7 (团结引擎)
- 角色渲染: Live2D Cubism SDK 5-r.4 (参数化变形 + 实时物理模拟)
- 3D 渲染: Unity Animator + FBX (预留混合渲染框架, 3D 模型尚未接入)
- AI 对话: DeepSeek Chat API + Function Calling (55 个法术工具, 9 个工具文件 + 7 个基础设施.cs)
- 视觉模型: GLM-4V (智谱) — 截图分析 + 多帧拼图动作视觉验证 (单模型)
- 本地决策: Ollama + qwen2.5:3b (MotionAgent 实时动作决策)
- 本地知识库: Ollama + nomic-embed-text (RAG 语义检索)
- JSON 序列化: Newtonsoft.Json (统一替换手写解析)
- 天气数据: wttr.in 默认 (cityCode=Nanjing) / QWeather 可选
- 文件搜索: Everything CLI (es.exe) / 递归回退
- 推送通知: Server 酱³
- 窗口管理: Win32 API (DWM / Shell_NotifyIcon / UIAutomation)
- 构建脚本: PowerShell + Unity Batchmode
- 平台: Windows 10/11 64 位

1.3 核心指标

- 渲染帧率: 20–60 fps (自适应三档降频)
- AI 响应延迟: 1–5 s (取决于网络与工具调用轮次)

- 内存管理：800 MB 触发 GC，1200 MB 强制 GC
- 系统内存监控：85% 预警 GC，93% 紧急降质
- 工具调用：55 注册工具（9 个工具文件，插件式自动发现），最多 10 轮回环（MAX_TOOL_ROUNDS=10）；N40 按意图注入工具子集（55→27）
- 空闲动作：9 种 JSON 配置驱动（7 参数化 + 2 硬编码特效）
- 运动记忆：上限 30 条，负反馈 10 条，无望检测 5 次
- 动作验证：GLM-4V 四帧拼图评分 + 三级测试验证
- 长期记忆：核心事实 + Top-5 重要 + 近期琐事，JSON 持久化

2 系统架构

2.1 分层架构

系统采用分层架构设计，从底层到上层依次为：

1	+-----+
2	UI 表示层
3	BallPanel(悬浮) RightPanel(右侧) ChatBubble
4	DebugWindow
5	+-----+
6	AI 核心层
7	ChatManager ToolCallInvoker IdleChatGenerator
8	MotionTranslator MotionAgent MotionVerifier
9	PersonalityManager
10	MotionMemoryManager VisionMotionVerifier
11	+-----+
12	感知层
13	KnowledgeBaseManager ActivityTracker BrowserTabReader
14	PetMemory TimeWeatherController PerformanceMonitor
15	GpuMonitor
16	+-----+
17	物理层
18	DesktopPet (重力/碰撞/地面检测) DragHandler
19	+-----+
20	渲染层
21	HybridRenderer -> Live2DRenderer / Model3DRenderer
22	+-----+
23	系统层

```
24 | WindowOverlay (DWM) | SystemTrayManager | ServerPoll |
25 +-----+
```

2.2 执行顺序

Unity 脚本执行顺序通过 `DefaultExecutionOrder` 属性严格控制：

1. `DesktopPet.Update()` (`order=0`) — 物理更新、状态转换、行走相位
2. `CubismPhysicsController.LateUpdate()` (`order=800`) — 衣服/头发/裙子物理模拟
3. `Live2DRenderer.LateUpdate()` (`order=801`) — 覆盖被物理重置的参数、空闲动画、交互反馈

顺序 801 确保所有 Live2D 参数在 Cubism Physics 运算后写入，避免物理覆盖关键参数。

2.3 依赖注入模式

组件间通过 Unity Inspector 序列化引用或单例模式连接。全局单例包括：`ChatConfig`（环境变量静态访问）、`PetMemory.Instance`（忆境）、`PetConfig.Instance`（天机簿）、`MotionAgent.Instance`、`MotionMemoryManager.Instance`、`GpuLoadMonitor.Instance`、`KnowledgeBaseManager.Instance`（藏书阁）。

3 渲染管道

3.1 Live2D Cubism 渲染

3.1.1 模型加载双保险

`Live2DRenderer` 使用两级加载策略：第一级加载 `AssetDatabase.LoadAssetAtPath`（Editor 模式），失败时第二级通过 `Resources.Load` 降级（构建模式）。

3.1.2 参数映射系统

`Live2DParameterMapper` 建立语义参数名到 Cubism 参数 ID 的双向映射，涵盖约 80+ 个参数：

```
1 mapper.RegisterParameter("ParamAngleX", "head_angle_x");
2 mapper.RegisterParameter("ParamBodyAngleX", "body_angle_x");
3 mapper.SetParameterValue("head_angle_x", 15f); // 头左倾 15 度
```

Listing 1: 参数映射示例

参数按身体部位分组：头部（3 轴旋转）、身体（3 轴倾角）、眼睛（开合/微笑/眼球）、眉毛（上抬/集中）、嘴巴（形态/张合）、手臂（左右多段）、呼吸。

3.1.3 Perlin 噪声驱动空闲微动

空闲状态由 Perlin 噪声驱动 7 组微动参数，使用不同的种子值确保互不关联：

$$\text{breath} = (\text{PerlinNoise}(t, 0) - 0.5) \times \text{Amplitude}$$

表 1: Perlin 噪声微动参数

参数	通道 (seed)	振幅
ParamBreath	(t, 0)	BREATH_AMPLITUDE
ParamBodyAngleX	(t, 1)	BODY_SWAY_X
ParamBodyAngleY	(t, 2)	BODY_SWAY_Y
ParamBodyAngleZ	(t, 3)	BODY_SWAY_Z
ParamAngleX	(t, 4)	HEAD_X
ParamAngleY	(t, 5)	HEAD_Y
ParamEyeBallX	(t+offset, 6)	EYE_X
ParamEyeBallY	(t+offset, 7)	EYE_Y

3.1.4 天气表情联动

天气状态自动影响空闲表情基调：

表 2: 天气表情联动

天气	表情效果
晴/多云	ParamMouthForm 微笑 +0.2
雨/阴	ParamBrowRY/LY 眉抬 +4, MouthForm 微嘟 +0.2
雷暴	同上 + ParamEyeLOpen 微睁
雪	ParamMouthOpenY 微张 +0.4, EyeLOpen 微睁 +0.2
夜晚 (18–22 点)	EyeLOpen 垂 +0.07
深夜 (22–5 点)	EyeLOpen 垂 +0.15

3.2 DWM 透明窗口

窗口透明使用 Windows DWM API 实现：

```

1 // 1. 设置窗口样式
2 SetWindowLong(hwnd, GWL_EXSTYLE, WS_EX_LAYERED | WS_EX_TRANSPARENT);

```

```

3
4 // 2. 扩展玻璃框架（黑色区域 = 透明）
5 MARGINS margins = new MARGINS { cxLeftWidth = -1 };
6 DwmExtendFrameIntoClientArea(hwnd, ref margins);
7
8 // 3. 设置背景纯黑透明
9 Camera.backgroundColor = new Color(0, 0, 0, 0);

```

Listing 2: DWM 玻璃层扩展

关键特性：

- 无绿边：DWM 玻璃层与纯黑背景配合，消除色键抠图的半透明边缘伪影
- 分层渲染：Live2D 渲染到 RenderTexture，通过 RawImage 显示在 Layer 31
- 点击穿透：每帧动态切换 WS_EX_TRANSPARENT
- DWM 崩溃保护：连续 5 次重建失败则跳过 DwmExtendFrameIntoClientArea

4 AI 对话系统

4.1 DeepSeek Chat API 集成

4.1.1 系统提示词注入

ChatManager.BuildSystemPrompt() 在每次对话前动态构建 System Prompt，注入内容包括：当前时间、法眼活动追踪摘要、浏览器标签页列表、长期记忆格式化文本、本地知识库（藏书阁）检索结果、Live2D 参数语义知识、具身动作生成能力描述。

4.1.2 Function Calling 架构

55 个注册工具（9 个工具文件）通过 JSON Schema 暴露给 DeepSeek，覆盖网页搜索、截图分析、系统控制、文件操作与搜索（Everything 毫秒级）、便签管理、课表查询、动作/表情播放、动作生成与视觉验证、本地知识库检索与索引、应用/文件夹启动、Pogget 集成、LaTeX 编译等。N40（T4）起按意图注入工具子集：首轮有意图 → 27 个相关工具，回环轮 → 已用工具 意图候选 CoreToolSubset（6 个）。

4.1.3 多轮工具调用循环

最多 10 轮的工具调用回环（MAX_TOOL_ROUNDS=10，非旧文档所述 5 轮）：

```

1 SendMessage()
2     SendRequestCoroutine()
3     1. 发送请求（含 system prompt + 历史）

```



```
4      2. 解析 response (Newtonsoft.Json)
5      3. 如果有 tool_calls:
6          检查 round < 10
7          遍历 tool_calls:
8              IsCoroutineTool()? -> 协程异步执行
9              同步工具 -> Execute()
10         收集结果, 回到 1 (下一轮)
11      4. 无 tool_calls -> 处理回复文本
```

Listing 3: 多轮工具调用循环

4.1.4 异步工具执行

7 个可能耗时的工具通过协程异步执行, 每帧检查 `task.IsCompleted`, 不阻塞主线程:

- `query_exams / query_scores / query_schedule / query_user_status` — HTTP GET 请求课表小程序服务端
- `search_files` — Everything CLI 全盘搜索 (毫秒级) 或递归回退
- `knowledge_search` — 本地知识库语义检索 (Ollama 嵌入 + 余弦相似度)
- `knowledge_index` — 本地知识库文件/文件夹索引 (Ollama 批量嵌入)

4.2 工具索引

表 3: 工具列表 (55 个, 9 个工具文件)

工具	功能
WebSystemTools.cs (14)	
<code>search_web</code>	联网搜索 (OpenClaw)
<code>open_url</code>	打开网页
<code>search</code>	本地搜索
<code>open_app</code>	通过 PATH/StartMenu
<code>open_folder</code>	打开资源管理器
<code>get_system_info</code>	获取系统信息
<code>lock_screen</code>	锁屏
<code>set_volume / mute</code>	音量控制
<code>get_mouse_pos</code>	获取鼠标坐标
<code>list_files</code>	列出目录文件

表 3 续前页

工具	功能
notify	推送通知
run_command	执行 CMD
power	关机/重启/睡眠
ClipboardFileTools.cs (14)	
get_clipboard / set_clipboard	剪贴板读写
get_weather	查询天气
file_open / file_move / file_copy / file_delete	文件操作
file_rename / file_info / file_create / file_read	文件操作
dir_create	创建目录
search_files / search_file	文件搜索 (Everything)
ReminderAcademicTools.cs (8)	
set_reminder / query_reminders / mark_reminder_done / delete_reminder	便签管理
query_exams / query_scores / query_schedule / query_user_status	学业查询
Live2DSyncTools.cs (7)	
set_expression	切换面部表情
play_action / stop_action	播放/停止动作
inspect_motion_memory / inspect_personality	自省
explore_body / control_body	身体参数探索/控制
VisionKnowledgeTools.cs (4)	
take_screenshot	截屏 + GLM-4V 分析
knowledge_search	藏书阁语义检索 (本地)
knowledge_index	索引文件/文件夹到藏书阁
openclaw_search	OpenClaw 搜索
MotionCoroutineTools.cs (5)	
generate_motion	LLM 生成自定义动作
explore_body_vision / run_verification / vis_verify / self_review	视觉验证/自评
其他 (3)	
launch_pogget	启动 Pogget
pogget_agent	Pogget Agent IPC (8)
compile_latex	LaTeX 编译 (经 OpenAI)

4.3 句子队列与优先级气泡

长回复被拆分为单个句子，以打字机风格逐句显示。ChatBubble 实现了三级优先级抢占机制：

表 4: 气泡优先级系统

优先级	用途	覆盖规则
Low	闲话、定时问候、待机气泡	可被任何高优消息覆盖
Normal	提醒、交互回应	不可被 Low 覆盖
High	AI 主动回复	不可被任何消息覆盖

4.4 自动闲聊系统

IdleChatGenerator 在无交互 60–120s 后触发, 通过 DeepSeek API 批量预生成 (每次 10 条), 本地 JSON 缓存。按时间周期自动刷新缓存, API 不可用时回退硬编码问候语。

5 具身动作系统

5.1 架构演进

动作系统经过持续演化:

- **Phase 1–5:** 硬编码空闲动作 (直接写在 Live2DRenderer 中)
- **Phase 6–7:** JSON 配置驱动 + IdleActionScheduler
- **Phase 8:** MotionTranslator — LLM 翻译自然语言到关键帧
- **Phase 9:** 闭环自评 — GLM-4V 视觉验证 + MotionMemoryManager
- **Phase 10 (V2.1):** MotionAgent — 自主动作决策 (4 级密度), 动作系统重构到 Live2DFramework/ActionAgent/
- **Phase 11 (V2.3+):** 移除 Qwen-VL, 多帧拼图评分; Newtonsoft.Json; GpuLoadMonitor; SafetyValidator

5.2 空闲动作系统

5.2.1 JSON 配置驱动

空闲动作定义在 idle_actions.json 中, 每个动作包含多阶段参数目标和插值曲线:

```
{
  "formatVersion": "v2",
  "actions": [{
    "id": 1,
```

```
    "name": "head_tilt",
    "displayName": "歪头",
    "weight": 10,
    "cooldown": 8.0,
    "phases": [
      { "duration": 0.5, "curve": "smooth",
        "targets": { "ParamAngleZ": 5 } },
      { "duration": 0.5, "curve": "smooth",
        "targets": { "ParamAngleZ": 0 } }
    ]
  }]
}
```

Listing 4: 空闲动作 JSON 配置

5.2.2 动作列表

表 5: 空闲动作列表

ID	名称	权重	类型
1	歪头	10	JSON
2	微笑	10	JSON
3	挑眉	8	JSON
4	星辉缠绕	5	硬编码
5	伸懒腰	8	JSON
6	委屈	6	JSON
7	法阵展开	4	硬编码
8	害羞	6	JSON
9	困惑	0	JSON（AI 仅用）

> N40 修正：空闲动作为 9 种（id 1-9）。旧文档所称爱心/哭/心跳（id 10-12）不在 idle_actions.json 中;ActionPresets/ 另有 11 个动作预设 JSON（blush/brow/confuse/cry/heart_e, 供工具与硬编码动作调用。

5.2.3 插值曲线

MotionGenerator 支持 6 种插值曲线：

linear: $f(t) = t$
smooth: $f(t) = 3t^2 - 2t^3$
easeOut: $f(t) = 1 - (1 - t)^2$
easeIn: $f(t) = t^2$
hold: $f(t) = 0$
bounce: 分段弹性曲线

5.3 LLM 动作翻译 (MotionTranslator)

5.3.1 核心流程

1. 构建 Body Schema — 所有可用参数名、范围、语义描述
2. 排除 BODY_MICRO 和 CLOTHES (截图不可见参数)
3. 注入运动记忆 — MotionMemoryManager 提供历史成功模板 + 负反馈
4. 调用 DeepSeek API (temperature=0.3, 超时 30s)
5. 参数富化: 纯头面参数自动注入肢体参数
6. 解析 JSON 响应 → MotionPlan (关键帧序列)
7. MotionGenerator 协程逐帧插值播放

5.3.2 SPECIAL PATTERNS

内嵌在 System Prompt 中的姿势模板, 帮助 DeepSeek 准确生成常见姿势 (N40 代码实测: 10 条, 9 个独特姿势 + 捂脸重复):

表 6: SPECIAL PATTERNS (10 条)

模式	关键参数
叉腰	arm_upper=15 25 (双臂), arm_lower=-15 -25 (肘弯)
行礼	body_angle_y=-15 -25 (前倾), arm_lower=-15 -25 (手下垂)
歪头思考	head_angle_z=15 25 (大角度), eye_ball_y=-0.5 -0.7
捂脸 (1)	arm_upper=20 30, arm_rotation=20 30, arm_reach=0.5 0.8, 低头
惊讶捂嘴	head_angle_y=8 15, eye_open=0.9 1.0, mouth_open_y=0.5 0.8
缩团	head_angle_y=-15 -25 (低头), 臂夹紧, body_angle_y=15 25
指	sword_finger_switch=1, finger_index=1.0, 其余 0
招手	arm_upper=20 28, finger_normal_1 2=0.6 1.0, 每 0.4s 交替
捂脸 (2)	同捂脸 (1) + hand_layer_100=1, hand_layer_120=0.8
合十祈祷	arm_upper=10 20, arm_lower=15 25, hand_layer_100=1, 低头

5.4 动作锁定机制

AI 生成动作播放期间，通过 `_aiControlLocked` 和 `_actionLocked` 双锁防止被空闲动作或走路覆盖：前者阻止 Perlin 噪声和天气表情更新，后者阻止空闲动作调度器和拖拽动画。

5.5 MotionAgent — 自主动作决策引擎

MotionAgent 是 V2.1 引入的常驻后台决策 Agent，在 ChatManager 不活动时自主决策动作：

- **决策循环**：等待间隔 → `ShouldDecide()` → `GatherContext()` → `DecideWithLLM()` / `FallbackDecide()` → `ExecuteDecision()`
- **4 级密度**：High (4s) / Med (8s) / Low (15s) / Sleep (30s)
- **本地 LLM**：Ollama qwen2.5:3b (LocalLLMClient, OpenAI-compatible API) → 连续失败 3 次后回退到概率模式
- **情绪状态机 (EmotionState)**：四维模型 Valence(-1 1)、Arousal(0 1)、Warmth(-1 1)、Energy(0 1)，指数衰减回归基线，半衰期 120s
- **自适应密度**：基于空闲时长、睡眠时间、用户专注进程自动调节
- **动作锁**：`_aiControlLocked` + `_actionLocked` 双锁防止被覆盖
- **全链路报告**：`GetPipelineReport()` 输出密度级别、回退模式、总动作尝试、成功/失败统计、Top-8 失败动作

5.6 MotionPlanner — 动作规划器

- **10 种硬编码模板**：挥手 (8 帧摆动)、点头 (5 帧)、摇头 (5 帧交替)、鞠躬 (3 帧)、伸懒腰 (4 帧)、叉腰 (3 帧)、捂脸 (3 帧)、指 (3 帧)、招手 (9 帧摆动)、合十 (3 帧)
- **6 个表情模板**：happy/sad/angry/surprised/sleepy/blush
- **3 阶段计划**：从默认 → 淡入 → 保持 → 回到默认
- **回退机制**：`GenerateGenericMotion()` 4 种变体 (纯头/抬臂/歪头 + 左手/微耸肩)

5.7 安全与 GPU 监控

5.7.1 SafetyValidator — 参数安全校验

- 互斥组: `eye_l_open` ↔ `eye_l_smile` 等不能同时设置
- 对称对: 5 对 (眼睛/眉毛), 检测单边设置不合理
- 极端值保护: 6 个参数 (眼闭合/嘴张大/头过度倾斜)
- `Validate()` 范围钳制 + 极端警告, `ValidateBulk()` 逐参数校验

5.7.2 GpuLoadMonitor — 游戏检测

通过 `ActivityTracker.CurrentCategory == "gaming"` 检测游戏:

- 进入游戏 → `LocalLLMClient.Paused = true` (暂停本地 LLM 决策)
- 退出游戏 → 30s 冷却后自动恢复
- 可选暂停 `MotionAgent` 本身 (`alsoPauseMotionAgent`)

6 闭环具身智能验证

6.1 架构演进

V2.3+ 对验证系统进行了重要精简:

- 移除 `Qwen-VL-Plus` (实测 401 Unauthorized 始终不可用)
- 引入 `ComposeCollage()` 多帧拼图: 在动作播放的 20%/40%/60%/80% 进度截图, 合成 2×2 拼图
- 单模型 GLM-4V 即可裁决, 节省约 50% API 调用时间

6.2 验证流程

完整的”生成 → 执行 → 观察 → 评估 → 学习”闭环:

DeepSeek(生成) → Live2D(执行) → 拼图 (四帧截图) → GLM-4V(评估) → 记忆 (学习)

6.3 VisionMotionVerifier — GLM-4V 多帧拼图评分

> **N40 修正**: 旧文档称 DualModelValidator (双模型), 实测该类 Qwen-VL 已删除、qwenScore 恒 0; N39 已简化回调签名 (4 参) 并删除恒 0 的 qwenScore 参数。实际为**单模型** VisionMotionVerifier (GLM-4V 付费模型, 180s 超时, 55% 截图采样)。

```

1 // 1. 动作播放期间在 20%/40%/60%/80% 进度截图
2 // 2. 合成 2x2 拼图
3 Texture2D collage = ComposeCollage(frames);
4 // 3. 发送给 GLM-4V 评分
5 // 4. 评分 >= passThreshold(3) -> 写入正反馈记忆
6 // 5. 评分 < 3 -> 触发负反馈

```

Listing 5: 多帧拼图验证核心

GLM-4V 评分标准:

$$\text{Score} = \begin{cases} 5, & \text{非常像, 动作精准传达意图} \\ 4, & \text{比较像, 主要特征符合} \\ 3, & \text{一般, 部分特征符合} \\ 2, & \text{不太像, 很多特征缺失} \\ 1, & \text{完全不像} \end{cases}$$

6.4 MotionMemoryManager — 运动记忆引擎 (闭环学习核心)

- **上限 30 条**: 按动作名索引, 每个动作仅保留一个最佳参数模板
- **高分覆盖低分**: GLM-4V 评分 $\geq$ 当前最佳时替换
- **负反馈系统**: 评分 ≤ 2 记入反例列表 (上限 10 条), 自动淘汰引用最少 + 最旧
- **无望检测**: 同一动作尝试 ≥ 5 次且最高分 ≤ 2 , 标记为「无望动作」优先淘汰
- **失败降权**: GetFailurePenalty() 持续低分降权 (0.3/0.4/0.7/1.0)
- **冷却机制**: 动作执行后 120s 内不注入 prompt, 防止复读
- **PhysicallyImpossibleActions**: 2 个已标记 (跳跃、转身背对)
- **独立 JSON 持久化** (Newtonsoft.Json), 不挤占 PetMemory 忆境

6.5 具身验证工具集

- **VisionMotionVerifier**: 10 个视觉测试 T1-T10 (害羞捂脸/挺胸叉腰/惊讶捂嘴/忧郁远望/俏皮眨眼/行礼/吓到缩团/骄傲抬头/歪头思考/合十祈祷)

- **MotionVerifier:** 三级测试 — Level 1 对照组 (C1-C5 挥手/点头/摇头/鞠躬/伸懒腰)、Level 2 测试组 (T1-T10)、Level 3 边界测试 (B1-B4 空描述/乱码/复杂/不可能)
- **ActionReferenceManager:** 持久化动作标准截图供 self-review
- **SelfTrainingManager:** 状态机驱动, 训练 7 个动作, 最多 3 次迭代

7 交互系统

7.1 拖拽系统 (DragHandler)

DragHandler 管理所有交互与点击穿透的协调。每帧根据鼠标位置和面板状态动态设置 `WS_EX_TRANSPARENT` 或清除该标记:

- 鼠标在 BallPanel 交互区域 → 关闭穿透 (允许点击面板)
- 鼠标在 RightPanel 交互区域 → 关闭穿透 (允许点击按钮/输入)
- 面板关闭时 → 强制重置穿透状态
- 拖拽释放时计算速度向量传递给 DesktopPet 物理引擎

拖拽释放时的抛掷物理:

$$\mathbf{v}_{\text{throw}} = \frac{\sum_{i=1}^n (\mathbf{p}_i - \mathbf{p}_{i-1})}{n \cdot \Delta t}, \quad |\mathbf{v}_{\text{throw}}| \leq \text{maxThrowSpeed} = 12$$

抛掷速度经多帧缓冲平均后限幅, 防止瞬间抖动的异常高速。

7.2 悬浮球辐射菜单 (BallPanel)

V2.1 全新引入的交互方式。桌面右下角粉色 悬浮球, 单击展开辐射菜单:

- **设置面板:** 任务权重滑块调节 + 保存/清空, 420×580px 浮动窗口
- **报告面板:** MotionMemoryManager 演武心经学习报告展示
- **便签面板:** 待办/已完成管理, 增删改查操作

面板通过 OnGUI 渲染, 使用自定义 GUIStyle。标题栏可拖拽移动, 右键关闭面板。PanelType 枚举管理三种面板状态 (None/Settings/Report/Reminders)。

7.3 右侧 Widgets 面板 (RightPanel)

Windows 11 Widgets 风格右侧滑出面板：

- ~ 键切换展开/收起，鼠标划过右边缘自动展开
- 1 秒自动隐藏延迟（鼠标离开后计时）
- 展开宽度 220px，收起宽度 8px，滑动速度 10f/s
- 4 个工具按钮：聊（聊天聚焦）/ 设（打开设置）/ 签（打开便签）/ 告（打开报告）
- 底部集成输入栏

`IsPointInInteractiveArea(Vector2)` 供 `DragHandler` 判断点击穿透状态。

7.4 分区点击反馈

通过 `CubismRaycaster` 实现分区检测：

- **Head** (高度 > 0.6)：摸头眯眼锁定动画
- **Body** (0.3 0.6)：捂胸惊讶
- **Leg** (< 0.3)：害羞踢腿

7.5 拖拽挣扎动画

拖拽过程中触发挣扎动画：双臂交替划水 + 双腿交替 + 身体扭动 + 慌张表情。帧间速度经平滑滤波后直接写入 `Cubism` 参数，绕过物理系统的 0.8s 延迟，实现即时响应。

拖拽时 20+ 头发输出参数由帧间速度直接驱动，裙子 (`Param82/87/84`)、法盘 (`Param49/51/57/60`) 同步跟随，实现物理惯性拖尾效果。

8 感知与记忆系统

8.1 法眼活动追踪 (ActivityTracker)

每 2 秒通过 `GetForegroundWindow()` 获取前台窗口标题，按关键词分类为编程、设计、游戏、学习、浏览、办公、通讯、音乐等。`BrowserTabReader` 通过 UI Automation API 读取主流浏览器标签页标题（Chrome/Edge/Firefox/Opera/Brave/Vivaldi）。

8.2 长期记忆系统 (PetMemory)

三层分层结构：核心事实（永久保留）、重要记忆（Top-5，按重要性排序）、近期琐事（时间衰减自动淘汰）。上限 30 条。反思反射：累积重要性 ≥ 30 时触发 LLM 提炼高层次洞察。同话题 120s 冷却。

8.3 人格演化系统 (PersonalityManager)

五维人格特质（勤勉/亲和/活泼/自信/求知）和三维关系模型（信任/亲密/熟悉度）随每轮交互动态微调。人格特质影响 EmotionState 的基线值、注入 SystemPrompt 引导 AI 语气和态度。长期无交互时缓慢回归中性。每 20 次交互触发人格里程碑记录。持久化至 pet_personality.json。

8.4 时间与天气系统 (TimeWeatherController)

天气源：wttr.in 默认（cityCode=Nanjing），QWeather 可选（需 Key）。（N40 修正：非旧文档所述“QWeather 优先”。）AI 天气语录：DeepSeek 批量生成符玄风格天气台词，按天气类型缓存。

8.4.1 AI 天气语录

由 DeepSeek 批量生成符玄风格的天气台词，按天气类型缓存，随天气变化自动刷新。

8.5 本地知识库（藏书阁）(KnowledgeBaseManager)

V2.4 引入的本地 RAG 系统，允许符玄语义检索本地文件内容。基于 Ollama 的 nomic-embed-text 嵌入模型（也可选 bge-m3），对指定文件夹进行自动分块索引。

- **分块策略**：以空行为段落分隔符，合并小段落至 800 字符上限，保留语义边界
- **嵌入计算**：调用 Ollama /api/embed 端点，批量化获取段落向量
- **语义搜索**：查询文本同样经 Ollama 嵌入后，与所有段落计算余弦相似度，返回 Top-5
- **支持格式**：.cs/.py/.js/.ts/.md/.txt/.json/.xml/.yaml/.html/.css/.tex/.lua 等 25+ 文件类型
- **自动注入**：每次对话后自动提取关键词后台检索，结果缓存至下一轮 System Prompt
- **工具集成**：knowledge_search（语义检索）和 knowledge_index（索引文件/文件夹）

- 持久化：索引数据（含原始文本和嵌入向量）保存至 `knowledge_base.json`
- 依赖：需安装 Ollama 并 `ollama pull nomic-embed-text`

8.6 核心物理模型

简化的 2D 像素物理引擎：

$$\begin{aligned}v_y &= v_y + g \cdot \Delta t \\y &= y + v_y \cdot \Delta t \\v &= v \cdot \text{AIR_RESISTANCE}\end{aligned}$$

8.7 地面任务状态机

5 种地面行为通过加权随机切换：MoveLeftEdge（权重 1）/ MoveRightEdge（权重 1）/ MoveLeftTime（权重 1）/ MoveRightTime（权重 1）/ StopTime（权重 6）。

8.8 多显示器支持

> N40 注记：旧文档称”通过 VirtualScreen 获取完整桌面区域，物理边界自动适配”，> 代码实测 WindowOverlay.isMultiMonitor 恒为 false（仅用 SM_CXSCREEN 主屏尺寸），> 跨屏行走/多屏边界适配为规划中能力（见 roadmap Phase 4.4）。

8.9 睡眠唤醒检测

帧间时间间隙检测： $\Delta t_{\text{gap}} > \text{THRESHOLD} \Rightarrow \text{RebuildDWM}() + \text{ResetPhysics}()$

9 窗口与系统集成

包含 DWM 透明窗口、系统托盘（Shell_NotifyIcon）、开机自启（注册表）、崩溃日志捕获（Application.logMessageReceived，自动截断 > 2MB）、进程互斥（Mutex DesktopPet_Unity_SingleInstance）。

9.1 崩溃日志系统

通过 Application.logMessageReceived 捕获异常和错误，追加到本地日志文件。日志长度超限时自动截断（保留后半部分），防日志爆炸。

9.2 进程互斥

使用 Mutex（DesktopPet_Unity_SingleInstance）防止多实例。

10 性能优化

10.1 三级性能档位

表 7: 性能档位

档位	目标帧率	RT 缩放	触发条件
High	60 fps	100%	FPS > 45 且 CPU/GPU < 80%
Normal	40 fps	75%	FPS 30–45 或 CPU/GPU < 85%
Low	20 fps	50%	FPS < 30 或 CPU/GPU > 92%

10.2 CPU/GPU/内存监控

- **CPU:** Win32 `GetSystemTimes` 获取总占用率
- **GPU:** NVML (NVIDIA Management Library)
- **系统内存:** `GlobalMemoryStatusEx`, 85% 预警 GC, 93% 紧急降质
- **应用内存:** `Profiler.GetTotalAllocatedMemoryLong`, 800MB 触发 GC, 1.2GB 强制 GC

11 便签与提醒系统

支持一次性/每日/工作日/每周四种重复规则。三级推送链路：气泡 → Windows Toast → Server 酱³。去重机制：通过 `HasPendingReminderContaining()` 关键词检查，防止 AI 设置与服务器推送重复。已完成任务独立视图 (N18 引入)。

12 构建与部署

标准构建使用 `build.ps1` (PowerShell + Unity Batchmode)，支持 `-Quick` 参数仅验证编译。构建后验证检查关键文件与日志关键字。

13 已知问题与 Bug (N38 审计 · N39 已全部修复)

N38 (2026-08-02) 代码真相审计确认 9 个已核实 Bug, N39 (同日) 已全部修复；另有三项结构性偏差中两项已接线修复：

表 8: 已知 Bug 与修复状态

编号	优先级	状态	问题
BUG-1	P0	已修复	<code>MotionAgent.IsSleepTime()</code> 恒返回 false (硬编码), 睡眠检测失效
BUG-2	P0	已修复	<code>ExecuteCombo</code> 在动作播放后才加 AI 锁 (锁时序错误)
BUG-3	P1	已修复	<code>GatherContext()</code> 注释称农历月相, 实际用公历 <code>now.Day</code>
BUG-4	P1	已修复	<code>ActivityTracker.UpdateInteractionTime()</code> 空实现
BUG-5	P1	已修复	<code>DualModelValidator</code> 名不副实 (Qwen-VL 已删, <code>qwenScore</code> 恒 0)
BUG-6	P2	已删除	<code>AutoMotionCollector</code> 死代码 ([Obsolete] 331 行)
BUG-7	P2	已修复	<code>MotionTranslator</code> 示例 <code>arm_right_upper: 0.8</code> 与规则 15–25° 矛盾
BUG-8	P2	已澄清	文档称 11 规则 + 12 特殊, 实际 10 规则 + 10 特殊
BUG-9	P2	已澄清	文档称 <code>BounceEaseOut</code> , 实际曲线名为 <code>Bounce</code>

另有三项结构性偏差:

- 反思回调 `ChatManager.OnReflectRequest` 恒为 null (反思未接线) → **N39 已修复**: `CheckReflection` → `DoReflection` (DeepSeek 提炼) → `CommitReflection` 接线, 死回调已删除
- `KnowledgeBaseManager.GetFormattedContext()` 为 STUB, 恒返回空字符串 → **N39 已修复**: 返回 `LastFormattedContext` 缓存 (`SearchAndFormat` 协程填充)
- 天气源默认 `wtr.in` (QWeather 为可选, 非优先) → **文档已修正**

N40 (2026-08-07) Token 优化 T1–T8 全部完成: 时间戳尾部缓存 (命中率 98.6%)、body schema 精简 (2543–5435 字符)、`max_tokens: 1200 + thinking disabled`、工具子集 55→27、历史 15000 字符预算 + Ollama 摘要、SystemPrompt 5012→2972 字符 (-41%)、Speculative Multi-Action、usage 日志。

14 版本路线图

表 9: 版本演进

版本	日期	主要变更
v0.2	2026-05	PNG 渲染初步 + API Key 环境变量
v0.3-v0.9	2026-06	空闲动作 / 右键菜单 / 物理 / DWM / 输入栏
N10-N18	2026-06	多轮工具 / 气泡 / 性能 / 推送 / 课表 / 文件搜索 / 去重 / 内存
V2.1	2026-07-09	悬浮球 + 辐射菜单; 动作系统重构到 ActionAgent/
V2.2	2026-07	悬浮球辐射菜单 + BallPanel 面板系统
V2.3	2026-07	报告面板复制按钮; MotionAgent 自主动作决策
V2.3+	2026-07-13	移除 Qwen-VL + 多帧拼图评分; Newtonsoft.Json 迁移; GpuLoad-Monitor 游戏检测; SafetyValidator 安全校验; 闭环学习 P0 修复; MotionAgent 架构清理
V2.4	2026-07-14	藏书阁—本地知识库 RAG 系统 (KnowledgeBaseManager) ; Ollama nomic-embed-text 语义检索; knowledge_search / knowledge_index 工具
N38	2026-08-02	代码真相审计: 52 工具 / 10 轮回环 / 10+10 规则 / Bounce 曲线 / wttr.in 默认 / 单 GLM 模型; 确认 9 个 Bug (IsSleepTime 恒 false、ExecuteCombo 锁时序、GetFormattedContext STUB、OnReflectRequest 恒 null 等)
N39	2026-08-02	修复全部 9 个 Bug: 真实睡眠判断 / ExecuteCombo 锁时序 / 农历月相 / 交互时间接线 / DualModelValidator 清理 / AutoMotionCollector 删除 / 示例角度 / 反思接线 / 知识库缓存注入
N40	2026-08-07	Token 优化 T1-T8 全部完成: 工具集 52→55 全量核实 / 工具子集按意图注入 55→27 / 历史 15000 字符预算 + Ollama 摘要 / SystemPrompt -41% / Speculative

15 附录

15.1 环境变量清单

表 10: 环境变量

变量名	必需	用途
DEEPSEEK_API_KEY	是	DeepSeek Chat API + MotionTranslator
GLM_API_KEY	是	GLM-4V 截图分析与多帧动作验证
QWEATHER_API_KEY	否	和风天气数据源
SERVERCHAN_SENDKEY	否	Server 酱 ³ 推送
BRIDGE_TOKEN	否	OpenClaw Bridge 鉴权 (64 字符; fallback 自动读 GATEWAY)

15.2 相关文件路径

表 11: 文件路径

用途	路径
长期记忆	D:/DesktopPetData/pet_memory.json
天机簿配置	D:/DesktopPetData/pet_config.json
人格状态	D:/DesktopPetData/pet_personality.json
提醒数据	D:/DesktopPetData/reminders.json
运动记忆	D:/DesktopPetData/motion_memory.json
验证日志	(不存在 — 旧文档误称 validation_log.json, 实际报告经 Debug.Log 输出)
活动记录	D:/DesktopPetData/activity_log.json
动作截图	D:/DesktopPetData/ActionRefs/
GLM-4V 拼图	D:/DesktopPetData/glm_collages/
本地知识库	D:/DesktopPetData/knowledge_base.json
LaTeX 输出	D:/DesktopPetData/Documents/
系统提示词	Assets/Resources/SystemPrompt.txt
空闲动作配置	Assets/Resources/Live2D/IdleActions/idle_actions.json
参数映射	Assets/Scripts/Live2DFramework/ParamMaps/fuxuan_map.json

15.3 关键设计决策

表 12: 设计决策

决策	理由
DWM 玻璃层而非色键	彻底解决半透明绿边问题
DeepSeek Chat（非本地）	高质量对话 + Function Calling 原生支持
GLM-4V 单模型（移除 Qwen-VL）	401 不可用 + 多帧拼图提升评分准确度
Newtonsoft.Json 统一序列化	替换手写 JSON 解析，提升可靠性
Ollama 本地 LLM 决策	低延迟实时动作决策，无需互联网
LLM 翻译关键帧	无限动作多样性 vs 硬编码有限集
JSON 配置驱动空闲动作	扩展性，无需改代码即可添加新动作
闭环具身验证	AI 动作能力持续自我优化
GpuLoadMonitor 游戏检测	避免 LLM 决策影响游戏性能
SafetyValidator 参数保护	防止 LLM 生成到物理极限值损坏模型
Ollama nomic-embed-text 嵌入	轻量级本地语义搜索，隐私数据不外传